

Чтобы Claude *помнил* ваш проект.

Сегодня учим Claude помнить правила, стиль и контекст вашего проекта — чтобы каждое утро не объяснять одно и то же. Файл `CLAUDE.md`, ссылки `@file`, и `auto-memory` в `~/.claude/memory/`.

ДЛИТЕЛЬНОСТЬ

5–7 дней · 2–4 ч/день

ЧТО ВЫ ОСВОИТЕ

`CLAUDE.md` · `@file` · `auto-memory` · многофайловые проекты

ГЛАВНЫЙ ИТОГ

Claude знает ваш проект, не задаёт повторных вопросов

АРТЕФАКТ

дневник инвест-операций и дашборд портфеля из брокерской выписки

Один skill для качественного UI — нужен на второй проект (дашборд). Если ставили в этапе 00, проверьте, что он на месте.

ОБЯЗАТЕЛЬНО

Из этапов 00–01

- **Node.js, Claude Code, VS Code** — те же, что были раньше.
- Привычка работать итерациями (этап 01).

ЖЕЛАТЕЛЬНО

Skill frontend-design

Что это: skill от Anthropic, который учит Claude делать узнаваемые, авторские интерфейсы — не «AI-усреднённые». Включает альтернативные эстетики (журнальная вёрстка, brutalism, retro-future, и т. д.).

Зачем сейчас: для дашборда портфеля — чтобы он выглядел как настоящий финансовый продукт, а не «типовая html-страница».

Проверить, не стоит ли уже: в Claude Code напечатайте `/skills` — `frontend-design` должен быть в списке.

```
# если ещё не стоит
$ npx @smithery/cli@latest skill add anthropics/frontend-design --agent claude-code
```

Где взять: smithery.ai/skills/anthropics/frontend-design.

Проверка: попросите Claude «сделай страницу в стиле финансовой газеты XX века» — результат должен быть характерным, а не «типичным».

Брокерская выписка в CSV

Что это: CSV-файл со списком ваших операций по инвестициям (покупки, продажи, дивиденды).

Зачем сейчас: на ней мы строим дашборд портфеля.

Где взять: экспорт из брокерского кабинета (Тинькофф, IBKR, Robinhood — везде есть «выгрузить отчёт за период в CSV»). Если у вас нет своего — не беда: на первом промпте попросим Claude *сгенерировать пример CSV* на 30 операций.

Папка проекта как «рабочее пространство»

До этого этапа мы работали так: «одна папка — одна задача» (карточка обо мне, калькулятор процента и т. д.). Теперь — «одна папка — многоэтапный проект, в который мы возвращаемся неделями».

Что это даёт:

- Claude видит *все ваши файлы сразу* и может между ними переключаться;
- правила проекта пишутся *один раз* — больше не нужно повторять «всегда категоризируй по X» в каждой сессии;
- проект можно отдать Claude «свежему» (новая сессия) — и он через 30 секунд снова в контексте.

Главный инструмент для этого — файл `CLAUDE.md` в корне проекта. О нём — следующая тема.

CLAUDE.md: правила, которые Claude помнит сам

`CLAUDE.md` — это обычный markdown-файл, который вы кладёте в корень папки проекта. Claude читает его автоматически в начале каждой новой сессии в этой папке.

Что туда писать:

- **Стиль и тон:** «комментарии на русском», «короткие имена переменных», «никаких лишних emoji».
- **Структура проекта:** где у нас данные, где код, где отчёты.
- **Правила работы:** «всегда добавляй дату в ISO», «не коммить файлы из `data/`».
- **Тематические допущения:** «валюта по умолчанию — рубль», «ставка налога — 13%».
- **Что не делать:** «не используй сторонние API без спроса», «не удаляй файлы старше 30 дней без подтверждения».

Главное правило: **пишите коротко и только то, чего нельзя угадать из кода**. Не пишите туда историю проекта, биографию или то, что и так очевидно.

Пример хорошего `CLAUDE.md` для инвест-проекта:

```
# Investing Journal — правила проекта

## Стек
- одностраничные html, без сборщиков
- chart.js через CDN, никаких других зависимостей

## Данные
- источник: data/operations.csv (date, ticker, side, qty, price, currency)
- валюта по умолчанию — USD
- даты в ISO: YYYY-MM-DD

## Стил
- editorial / paper-and-ink, тёплая палитра (как сайт курса)
- засечковый шрифт, IBM Plex Mono для цифр

## Правила
- НЕ добавляй emoji в код
- комментарии на английском, тексты UI на русском
- перед удалением файлов из data/ — спрашивай
```

@-ссылки: указать Claude конкретный файл

Если в вашем проекте десятки файлов, Claude может тратить время на их «сканирование» при каждой задаче. Когда вы уже знаете, какой именно файл нужен, проще указать его напрямую через `@`:

```
> посмотри @data/operations.csv и собери short-отчёт по тикерам.
> используй формат, описанный в @docs/report-template.md.
```

Что Claude делает:

- Прочитает указанные файлы *сразу*, без сканирования папки;
- Не будет «угадывать», какие ещё файлы могут пригодиться;

- В ответе явно укажет, что использовал именно эти.

Удобно для:

- длинных проектов с десятками файлов;
- задач, где важна точность («только этот CSV, не трогай остальные»);
- повторяющихся задач — быстрее, чем заново описывать структуру.

Auto-memory: что Claude помнит между сессиями

`CLAUDE.md` живёт в *проекте* — его читают только когда работают именно с этой папкой. Auto-memory — *глобальная*: всё, что Claude в ней запомнил, доступно в любой папке, любой сессии.

Файлы лежат в `~/.claude/memory/`. Главный из них — `MEMORY.md`, индекс. Туда Claude сам пишет, когда узнаёт что-то полезное на будущее.

Четыре типа записей:

1. **user** — о вас: роль, опыт, предпочтения. «*Пользователь — финансовый аналитик, любит таблицы, не любит длинные комментарии*».
2. **feedback** — ваши указания, как работать или как НЕ работать. «*Не используй тоск-данные для интеграционных тестов — в прошлый раз пропустили баг*».
3. **project** — контекст текущих работ: дедлайны, инициативы, причины решений. «*Переход с старого API — до 1 июня, по требованию compliance*».
4. **reference** — «где искать»: ссылки на Linear, Jira, Slack, дашборды. «*Пайплайн-баги — в Linear-проекте INGEST*».

Полезные команды:

```
# посмотреть, что Claude помнит
> покажи мою auto-memory

# попросить запомнить
> запомни: я предпочитаю tabular layout для финансовых отчётов

# попросить забыть
> забудь, что я говорил про предпочтение dark theme
```

Важное: Claude *сам решает*, когда писать в auto-memory. Но вы можете прямо попросить.

Чтение существующего проекта: «расскажи, что тут происходит»

Когда вы открываете *чужой* проект (или свой, в который не заглядывали полгода), не нужно «вчитываться вручную». Запустите Claude и попросите ввести в курс:

```
> расскажи, что это за проект:
> – что он делает (одна-две фразы),
> – какая структура папок,
> – какие главные файлы,
> – что нетипично, на что мне стоит обратить внимание.
```

Claude пройдётся по файлам и даст вам краткий «бриф». Это особенно полезно перед тем, как:

- добавлять новую функцию в старый проект;
- возвращаться к курсу после паузы;
- взять чужой код (например, скачали с GitHub) и хотите понять, что там.

Multi-file edits: правки в нескольких файлах сразу

В многофайловом проекте часто одно изменение влияет на несколько файлов: переименовали колонку в CSV — нужно поправить и парсер, и дашборд, и документацию.

Claude умеет делать это одной просьбой:

```
> в operations.csv поле «side» переименовалось в «direction».  
> найди все упоминания «side» во всех файлах проекта и обнови.  
> покажи список изменений перед применением.
```

Чем такой подход лучше «по файлу за раз»:

- Claude видит *все связи* — не забывает обновить какой-то файл;
- можно потребовать *«покажи план перед применением»* — и сначала посмотреть список;
- откат проще: «верни всё, что ты только что изменил».

Хорошая привычка: для крупных правок ставить plan mode (этап 00) или просить «сначала покажи список изменений, я утвержу — потом применяй».

Превращаем калькулятор из этапа 01 в «проект»

Возьмите папку `compound` из этапа 01. Там у вас три файла: `compound-classic.html`, `compound-dark.html`, `compound-print.html`. Сейчас сделаем из них «настоящий проект».

PRACTICUM

Прогон · CLAUDE.md, @-ссылки, multi-file edit

1. Зайдите в папку `compound`, запустите `claude`.
2. Попросите Claude составить `CLAUDE.md`:

```
> составь файл CLAUDE.md в корне этого проекта.  
> зафиксируй: стек (плоский html, chart.js через CDN),  
> стиль (минимализм, тёплая бумажная палитра),  
> правила (без emoji, всё одним файлом, открывается двойным кликом),  
> что у меня сейчас три варианта: classic / dark / print.
```

3. Откройте `CLAUDE.md` в VS Code, прочитайте, поправьте если нужно.
4. Закройте Claude (`/exit`), откройте заново. Попросите: «добавь в *dark-версию* кнопку «сбросить» рядом с «посчитать»». Заметьте: Claude сразу знает, какой стиль вам нужен — он прочитал `CLAUDE.md` при старте.
5. Попробуйте **multi-file edit**:

```
> во всех трёх файлах compound-*.html  
> добавь сверху ссылку «← к остальным версиям»,  
> которая переключает между classic / dark / print.  
> покажи план перед применением.
```

6. Утвердите план, дайте применить, откройте все три файла — должна появиться навигация.
7. Попробуйте **@-ссылку**:

> глядя на @compound-classic.html, опиши его в одном абзаце –
> так, как написал бы дизайн-критик.

8. Финал: попросите *«запомни на будущее: я предпочитаю тёплые палитры с охряным акцентом»*. Это уйдёт в `~/ .claude/memory/`. На следующих этапах увидите, как Claude учитывает это в других проектах.

Что мы тренируем: **писать** `CLAUDE.md`, **работать с несколькими файлами одной просьбой**, **попадать в файл точно через** `@`, **передавать заметку в auto-memory**.

Дневник операций и дашборд портфеля

Оба проекта используют один и тот же набор данных — ваш CSV с инвестиционными операциями. Если своего нет — первый проект сгенерирует тестовые данные, и второй сможет на них работать.

Дневник инвестиционных операций

Цель: многофайловый проект с `CLAUDE.md`. Хранилище — CSV. Веб-страница для добавления операций (покупка/продажа/дивиденд) и их просмотра с фильтрами.

Шаги

1. Создайте папку `portfolio-journal` в `claude-course`, запустите там `claude`.
2. Сначала — `CLAUDE.md`:

```
> составь CLAUDE.md для этого проекта:  
> - стек: одна html-страница + один csv-файл рядом, без бэкенда  
> - данные: data/operations.csv с колонками  
>   date, ticker, side (buy/sell/dividend), qty, price, currency, note  
> - стиль: editorial, тёплая палитра, IBM Plex Mono для цифр  
> - правила: даты ISO, currency по умолчанию USD, без emoji
```

3. Сгенерируйте тестовые данные (если нет своих):

```
> создай data/operations.csv с 30 реалистичными операциями  
> за последние 12 месяцев. шесть тикеров (AAPL, MSFT, GOOGL,  
> NVDA, BRK.B, VOO), buy/sell/dividend смешаны.
```

4. Сделайте страницу-просмотрщик:

```
> сделай index.html — страница «дневник операций».  
> читает data/operations.csv (через fetch), показывает таблицу  
> с сортировкой по дате (новое сверху).  
> фильтры сверху: тикер (выпадашка), тип операции, период.  
> форма «добавить» внизу — сохранение в localStorage,  
> отдельная кнопка «выгрузить в csv» — сливает данные обратно в файл.
```

5. Откройте через локальный сервер (или попросите Claude поднять `python -m http.server`). Поработайте с интерфейсом.

6. Уточните то, что не понравилось:

- «сделай строки таблицы зеброй для удобства»;
- «добавь итог снизу: общее количество операций, сумма покупок и продаж»;
- «выделяй *sell* красным, *buy* зелёным, *dividend* охряным».

7. Финальный аудит: «перечисли в виде списка, что ты принял за меня — форматы, валюту, цвета, чего нет в *CLAUDE.md*».

Что вы здесь освоили

- писать `CLAUDE.md` для нового проекта;
- попросить Claude сгенерировать тестовые данные под вашу схему;
- многофайловый проект (html + csv) с импортом-экспортом данных;
- аудит с учётом того, что в `CLAUDE.md` УЖЕ есть.

Дашборд инвестиционного портфеля

Цель: страница-дашборд, которая читает `operations.csv` из проекта A и показывает: текущие позиции по тикерам, P&L, круговую диаграмму распределения, тренд капитала по месяцам.

Шаги

1. В той же папке `portfolio-journal` запустите Claude.
2. Включите `frontend-design` skill через явное упоминание:

```
> используя skill frontend-design, сделай dashboard.html –  
> финансовый дашборд по моему портфелю.  
> данные: @data/operations.csv (та же схема, что в проекте A).  
> разделы:  
> 1) текущие позиции (тикер, qty, средняя цена, оценка);  
> 2) P&L по позициям (реализованный и нереализованный);  
> 3) круговая диаграмма распределения капитала по тикерам;  
> 4) график капитала по месяцам.  
> стиль – как страница из financial times, тёплый кремовый,  
> засечковый шрифт для заголовков, IBM Plex Mono для цифр.  
> chart.js через CDN, без других зависимостей.
```

3. Откройте `dashboard.html`. Прокрутите все разделы. Проверьте формулы:
 - текущая позиция = сумма всех buy minus сумма всех sell, по тикеру;
 - средняя цена = взвешенное среднее по buy-операциям;
 - оценка = текущая позиция × последняя цена в CSV (или ввод вручную, если нет «текущей»).
4. Уточняйте:
 - «в P&L раздели реализованный (по закрытым позициям) и нереализованный (по открытым)»;
 - «в круговой диаграмме сделай легенду справа, цвета пастельные»;
 - «в графике капитала добавь линию «вложено» и линию «оценка» — разница и есть P&L»;
 - «добавь сверху три KPI-плитки: общая стоимость, P&L, доходность за год в %».

5. Зафиксируйте изменения в `CLAUDE.md` :

```
> обнови CLAUDE.md: добавь раздел «дашборд» –  
> описание секций, формулы, цвета операций (buy/sell/dividend).  
> чтобы при следующих доработках Claude помнил, как тут устроено.
```

6. Финальный аудит: *«какие три самых важных предположения ты принял про мои данные? что ломается, если они окажутся не такими?»*

Что вы здесь освоили

- задействовать установленный skill в промпте по имени;
- дашборд с графиками над живыми CSV-данными;
- обновлять `CLAUDE.md` по ходу — чтобы он «взрослел» вместе с проектом;
- аудит-вопрос про предположения о данных, не о дизайне.

Оба самостоятельных — вариации проектов А и В. Структура та же: новая папка, `CLAUDE.md`, тестовые данные через Claude (если нет своих), просмотр + дашборд. Никаких новых концепций.

Журнал пассивного дохода

Цель: те же три файла (CLAUDE.md + index.html + data/income.csv), но для учёта дивидендов, купонов по облигациям и поступлений по ETF. Просмотр с фильтрами по инструменту.

Подсказки

- Колонки CSV: `date, instrument, type (dividend/coupon/distribution), amount, currency, note`.
- В CLAUDE.md обязательно укажите валюту по умолчанию и что считать «годом» (календарный или fiscal).
- На странице помимо таблицы — общий итог за период и средний месячный доход.
- Фильтры: тип поступления, период, инструмент.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

— сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ в корне есть `CLAUDE.md`, в нём явно указано: схема данных, валюта, что считать «годом», правила;
- ✓ есть тестовые или ваши данные в `data/income.csv`, минимум 20 записей;
- ✓ таблица сортируется и фильтруется без перезагрузки страницы;
- ✓ сверху виден итог за выбранный период и средний месячный доход;
- ✓ после `/exit` и нового запуска `claude` Claude мгновенно «в курсе» проекта (не задаёт вопросов про схему);
- ✓ вы провели хотя бы один аудит-промпт.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Открой новую сессию в этой папке. Прочитай CLAUDE.md. Расскажи мне, как ты понимаешь этот проект, в трёх предложениях. Если CLAUDE.md написан хорошо — ты опишешь его точно. Если что-то размытое — это и есть пробел, который нужно дополнить».

Дашборд P&L по закрытым сделкам

Цель: вторая страница в папке `portfolio-journal` — `trades.html`.

Не дублирует основной дашборд, а добавляет: «закрытые сделки» (buy + соответствующий sell по тому же тикеру), реализованный P&L, лучшие/худшие сделки.

Подсказки

- Используйте *тот же* `data/operations.csv` — не создавайте новый.
- Алгоритм закрытия сделки: пара buy + sell по тому же тикеру в порядке FIFO.
- Реализованный P&L = сумма по каждой закрытой сделке.
- Перед началом обновите `CLAUDE.md` — добавьте раздел про `trades.html`.
- На странице — таблица закрытых сделок, плюс две KPI: лучшая сделка и худшая.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

— сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ `trades.html` работает на тех же данных, что `dashboard.html`;
- ✓ в `CLAUDE.md` описано, что такое «закрытая сделка» и по какой логике (FIFO);
- ✓ таблица показывает по каждой закрытой сделке: тикер, qty, цена in, цена out, P&L, % к входу;
- ✓ есть KPI «лучшая сделка» и «худшая» с тикером и %;
- ✓ навигация между `index.html`, `dashboard.html`, `trades.html` сделана через multi-file edit одной просьбой;
- ✓ проект остался многофайловым «единым целым», а не тремя независимыми страницами.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Прогони мой `trades.html` на тестовом сценарии: я купил 10 AAPL по 100, затем 5 по 120, потом продал 8 по 150. Сколько закрытых сделок будет в FIFO? Какой реализованный P&L? Сверх с тем, что показывает страница».

Что добавилось к словарю

Восемь новых терминов — вокруг памяти и многофайловых проектов.

Подчёркнуты пунктиром в тексте, наведите/коснитесь — всплывёт справка.

CLAUDE.md	Markdown-файл в корне папки проекта. Claude автоматически читает его в начале каждой сессии и применяет как «правила проекта»: стек, структуру, стиль, ограничения. Главный способ передать Claude <i>контекст без повторения</i> .
@-ссылка	Точечная ссылка на файл в промпте: @data/operations.csv. Claude читает именно его, не сканируя всю папку. Удобно в больших проектах и для точности.
Auto-memory	Долгосрочная память Claude, общая для всех проектов. Хранится в ~/.claude/memory/. Claude сам решает, что туда записать — но вы можете попросить «запомни...» или «забудь...».
Тип памяти (user / feedback / project / reference)	Четыре категории auto-memory: user — о вас, ваших предпочтениях. feedback — ваши указания, как работать. project — контекст текущих инициатив. reference — «где искать» (Linear, Slack, дашборды).
Multi-file edit	Изменения в нескольких файлах одной просьбой. Claude видит все связи и обновляет файлы согласованно. Обычно лучше попросить «покажи план перед применением».
CSV	Comma-separated values — «таблица в текстовом файле». Каждая строка — запись, поля разделены запятой. Открывается в Excel или Google Sheets, но при этом видна «как есть» в любом редакторе.
Дашборд	Одна страница с ключевыми показателями и графиками — чтобы охватить состояние «одним взглядом». В этом этапе — дашборд по инвестиционному портфелю.
P&L (Profit and Loss)	Прибыли и убытки. <i>Реализованный</i> P&L — по закрытым сделкам (продали с плюсом или минусом). <i>Нереализованный</i> — по открытым позициям (на бумаге).
FIFO	First-In-First-Out — «первым купил, первым продал». Стандартный способ закрытия позиций при подсчёте P&L: первая sell-операция «закрывает» самую старую buy.

Семь вопросов на закрепление

Сначала ответ — потом раскрывайте «правильный». После сравнения отметьте: *знал* / *не знал*. По итогам подскажу, что повторить.

01 Зачем нужен файл `CLAUDE.md` в корне проекта?

Это «правила проекта»: стек, структура папок, стиль, ограничения, тематические допущения. Claude читает его автоматически в начале каждой сессии в этой папке — не нужно повторять одно и то же. Главное: писать коротко и только то, что нельзя угадать из кода.

02 Чем `CLAUDE.md` отличается от auto-memory?

`CLAUDE.md` — локально, в конкретной папке. Auto-memory (`~/.claude/memory/`) — глобально, во всех ваших проектах. Правило: специфика проекта — в `CLAUDE.md`, общая информация о вас и ваших привычках — в auto-memory.

03 Когда уместно использовать `@-ссылку` на файл в промпте?

Когда вы точно знаете, какой файл нужен Claude — и не хотите, чтобы он сканировал всю папку. Особенно полезно в больших проектах и для задач, где важна точность («только этот CSV, не трогай остальные»).

04 Какие 4 типа записей бывают в auto-memory? Кратко — что в каждом.

user — о вас, роль, опыт, предпочтения. **feedback** — ваши указания «как» и «не как» работать. **project** — контекст текущих инициатив (дедлайны, причины). **reference** — «где искать» (Linear, Slack, дашборды).

05 Что такое multi-file edit и чем он лучше «по файлу за раз»?

Изменения в нескольких файлах одной просьбой. Claude видит все связи и обновляет согласованно — не забывает какой-то файл. Можно потребовать «покажи план перед применением». Откат тоже проще: «верни всё, что только что изменил».

06 Чему равен реализованный P&L, если я купил 10 акций по 100 и продал их по 120, а ещё 5 по 120 у меня всё ещё в портфеле?

Реализованный P&L = $(120 - 100) \times 10 = \mathbf{200}$. Только закрытые сделки. Открытые 5 акций по 120 в P&L не идут — они «нереализованные».

07 Как проверить, что ваш `CLAUDE.md` «достаточен» — то есть Claude точно понимает проект?

Закрыть Claude (`/exit`), открыть свежую сессию в той же папке, попросить: «*опиши проект в трёх предложениях*». Если описание точное и конкретное — `CLAUDE.md` хорош. Если размыто или Claude задаёт уточняющие вопросы — добавьте в `CLAUDE.md` то, что не хватало.

Что я теперь умею

Отметьте уверенно освоенное. Сомневаетесь — вернитесь к соответствующей теме. Этап готов закрыть, когда все галочки стоят.

- ☐ Пишу `CLAUDE.md` для нового проекта — стек, данные, стиль, правила.
 - ☐ Использую `@`-ссылки, чтобы указать на конкретные файлы.
 - ☐ Знаю, что такое auto-memory и четыре типа записей; могу попросить «запомни» / «забуди».
 - ☐ Делаю multi-file edits одной просьбой с «покажи план перед применением».
 - ☐ Прошу Claude «расскажи, что тут происходит» в незнакомом проекте.
 - ☐ Делаю проект из нескольких файлов с CSV-данными и графиками.
 - ☐ Сделал четыре проекта: дневник операций, дашборд портфеля, журнал пассивного дохода, P&L по закрытым сделкам.
-
-

Готовы закрыть этап? Нажатие фиксирует прогресс на главной странице.

«Хороший проект — тот, к которому Claude может вернуться через месяц и не спросить, что тут происходит».

Этап 02 пройден — Claude теперь знает ваш проект. Дальше — учим его ходить в интернет и использовать готовые skills для живых данных и дизайна.

ВНУТРИ ЭТАПА

- § 0 · Подготовка
- § I · Темы
- § III · Проекты
- § V · Словарь
- § VI · Quiz
- § VII · Чек-лист

КУРС

- К оглавлению
- ← Этап 01
- Этап 03